

HANDS-ON TECHNICAL DEEP DIVE

Retrieval-Augmented Generation

From Concepts to a Working End-to-End System

Chunking

Embeddings

FAISS & Chroma

Hybrid Retrieval

Evaluation

End-to-End Build

RR

Ruthran Raghavan

Chief AI Scientist

ruthranraghavan.com

Why Retrieval-Augmented Generation?



Hallucination

Confident, fluent answers that are simply wrong.



Stale Knowledge

The model only knows what was in its training data.



No Private Data

Can't see your internal docs, tickets, or wikis.



RAG grounds every answer in retrieved facts



Retrieve

relevant passages from your knowledge base



Augment

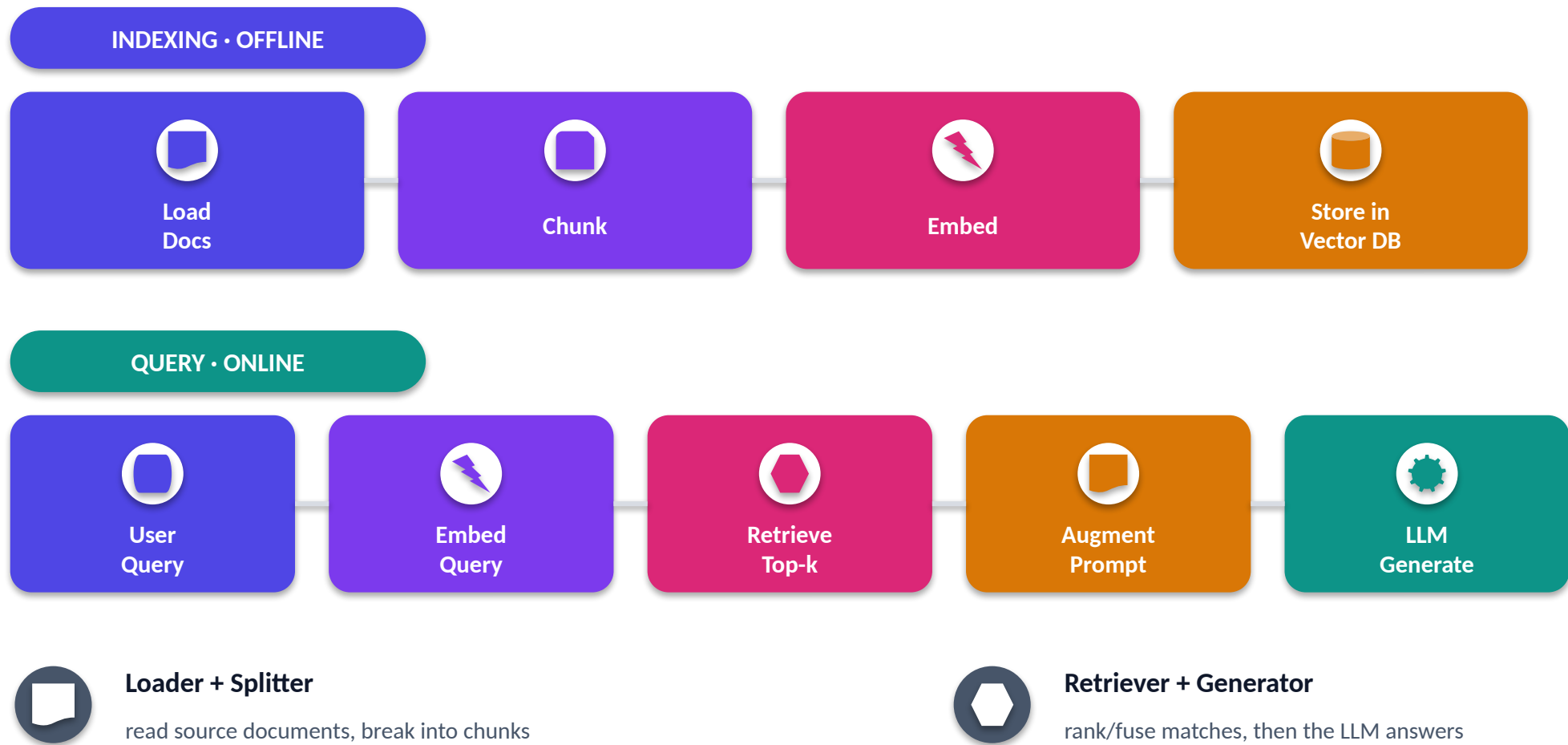
the prompt sent to the LLM with that context



Generate

an answer constrained to what the context supports

RAG Architecture: Two Pipelines



Chunking Strategies & Trade-offs



Fixed-Size

Every N characters.
Fast, but can cut
mid-word/sentence.



Fixed + Overlap

Overlapping windows.
Keeps boundary context,
adds redundancy.



Sentence-Based

Groups whole sentences.
Never splits mid-sentence,
variable size.



Recursive

Paragraph -> sentence
-> word fallback.
Most common default.



Token-Based

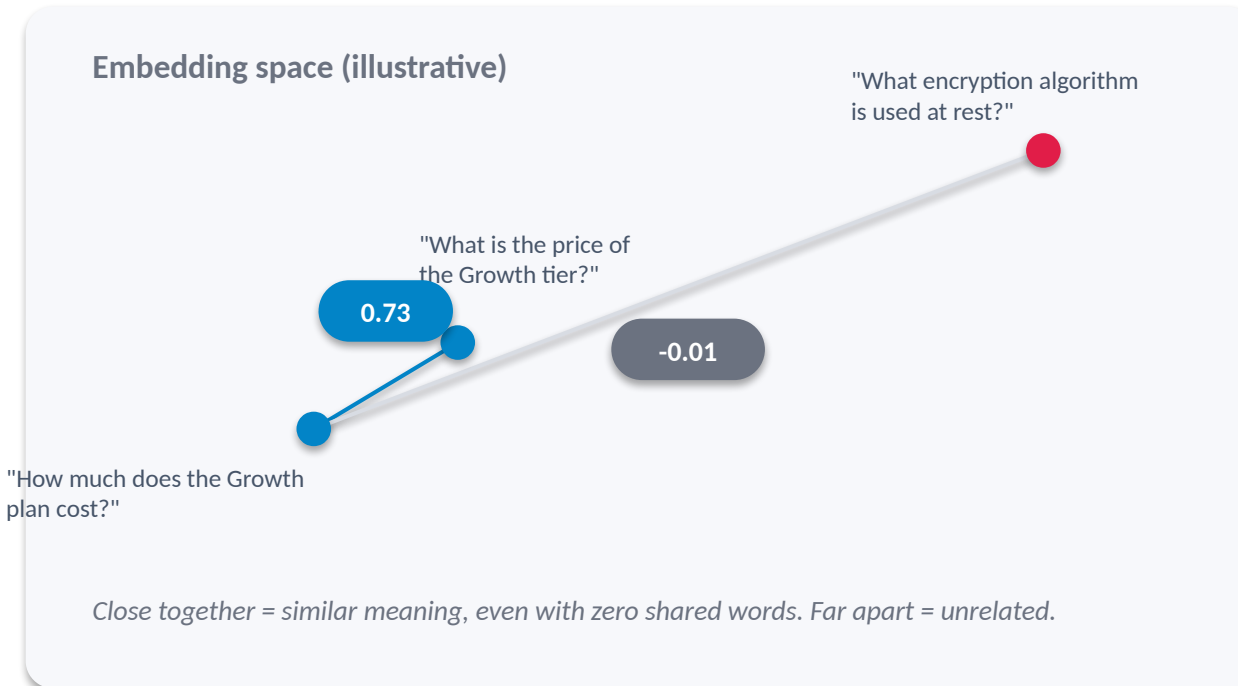
Splits by token count.
Matches model context
limits precisely.



Rule of thumb

start with recursive chunking + a small overlap; tune chunk size to your embedding model's sweet spot

Embeddings: Turning Text into Meaning



384-1536 dimensions

typical embedding vector size



Cosine similarity

the standard distance metric used



Open-source or API

MiniLM/BGE/E5 locally, or OpenAI/Cohere



Same model, both sides

must embed docs & queries identically

Vector Databases: FAISS vs Chroma



FAISS

A similarity-search library, not a server
Manual persistence - you save/load the index
Build your own metadata filtering
Index types: Flat, IVF, HNSW, PQ, and more
Best for: maximum control, custom infra, huge scale



Chroma

An embedded, all-in-one vector database
Persistence built in (SQLite/Parquet-backed)
Metadata filtering built in out of the box
HNSW approximate search under the hood
Best for: fast prototyping, small-to-mid scale



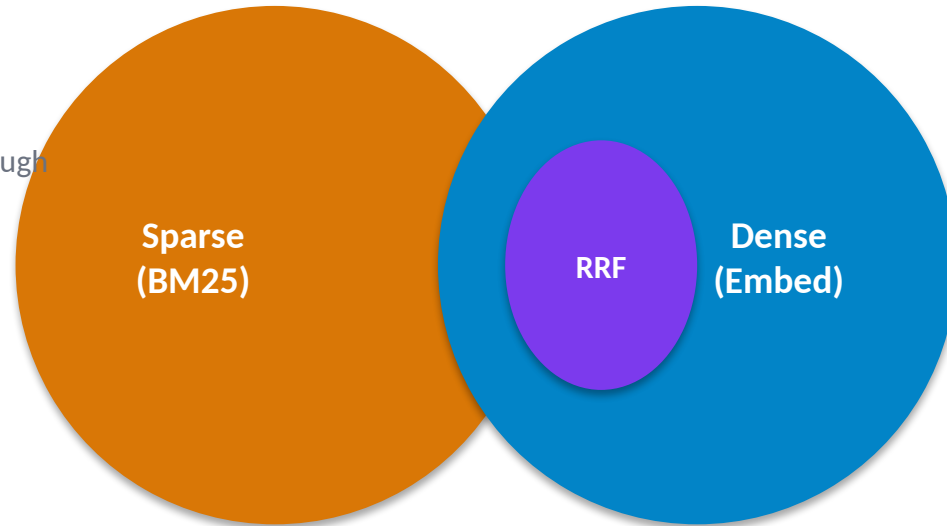
Same embeddings, matching results

both stores return identical top matches when indexed with the same vectors

Hybrid Retrieval: Sparse + Dense

$$\text{score} = \sum 1/(k+\text{rank})$$

Reciprocal Rank Fusion blends both rankings by position - robust even though BM25 and cosine scores live on different scales.



Exact terms, numbers,
acronyms, product codes

Meaning & paraphrase,
synonyms, natural language



One retriever, both styles

handles keyword queries and paraphrased questions equally well



Same-scale fusion

rank-based scoring means no manual weight tuning between retrievers

Evaluating a RAG System

RETRIEVAL METRICS



Precision@k

% of top-k results that are relevant



Recall@k

% of all relevant docs found in top-k



MRR

1 / rank of the first relevant result



nDCG@k

rewards relevant results ranked higher

GENERATION METRICS



Faithfulness

is the answer grounded in the retrieved context (no hallucination)?



Answer Relevance

does the answer actually address the question asked?



Context Precision/Recall

did retrieval surface the right context at all?

TOOLING

RAGAS

Azure AI Foundry Eval

LLM-as-judge

Langfuse

Arize Phoenix

End-to-End RAG Implementation



One pipeline class

wires chunking, embeddings, indexing, and retrieval together



Recursive chunking

sentence-transformer embeddings, FAISS + BM25 side by side



Hybrid retrieval (RRF)

selects the top-k chunks most relevant to the question



Cited, grounded prompt

the LLM must answer only from the numbered context



Standard chat completion

generation calls any LLM API (e.g. Azure OpenAI, OpenAI)



Fails gracefully

retrieval still runs and displays results if generation is unavailable

Key Takeaways



RAG grounds LLMs

Retrieval + Augmentation + Generation using your own data



Chunking quality matters

strategy & size directly shape retrieval quality



FAISS or Chroma

control-at-scale vs. speed-to-prototype - both are valid



Hybrid beats either alone

sparse + dense fusion handles varied query styles



Evaluate both layers

retrieval metrics and generation metrics, continuously



Ship end-to-end

one pipeline: load, chunk, embed, retrieve, cite, generate

Thank you - questions?

Ruthran Raghavan | Chief AI Scientist | ruthranraghavan.com